

[CS230] Foundations of Neural Networks: Python & Vectorization

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network

- 2.1 Architecture & Forward Propagation - *Completed*
- 2.2 Cost Function & Gradient Descent - *Completed*
- **2.3 Python & Vectorization (Current Unit)**
 - * Overview: Why Vectorization?
 - * SIMD: The Hardware Magic
 - * Broadcasting Rules
 - * Implementation & Benchmark

Chapter 3. Shallow Neural Networks - *Upcoming*

지난 시간 복습 및 연결

우리는 지난 시간까지 딥러닝의 '이론적 토대(비용 함수, 경사 하강법)'를 완성했습니다. 이론적으로는 완벽합니다. 하지만 이 수식을 컴퓨터에게 그대로 주면 학습하는 데 수십 년이 걸릴지도 모릅니다. 이제 이 이론에 '제트 엔진'을 달아줄 시간입니다. 초심자와 전문가를 가르는 가장 결정적인 기술, **벡터화(Vectorization)**를 배워봅시다.

1 Unit Overview

□ 핵심 목표

이 단원은 딥러닝 코드를 수백 배 빠르게 만드는 '**최적화 기술**'을 다룹니다.

- **개념:** 'for-loop'를 죄악시하고, 행렬 단위 연산(Vectorization)을 해야 하는 이유를 배웁니다.
- **원리:** CPU/GPU의 SIMD(병렬 처리) 아키텍처가 어떻게 연산을 가속하는지 이해합니다.
- **기술:** NumPy의 핵심 기능인 **브로드캐스팅(Broadcasting)**의 규칙과 위험성을 파악합니다.
- **검증:** 실제 코드로 100만 개의 데이터를 연산해보며 속도 차이를 눈으로 확인합니다.

2 Essential Terminology

용어	설명	한 줄 핵심 요약
Vectorization	벡터화	반복문 없이 데이터를 통째로(행렬로) 연산하는 기법
SIMD	Single Instruction, Multiple Data	명령어 하나로 여러 데이터를 동시에 처리하는 CPU 기
Broadcasting	브로드캐스팅	모양이 다른 배열끼리 연산할 때 자동으로 크기를 맞춰
NumPy	넘파이	파이썬의 느린 속도를 C언어 레벨 최적화로 극복한 수
Rank-1 Array	랭크-1 배열	‘(5,)’ 처럼 행도 열도 아닌 애매한 배열 (버그의 주범)

3 Core Concepts: 속도의 비밀

3.1 1. Vectorization (벡터화란 무엇인가?)

한 줄 요약: 하나씩 처리하지 말고, 트럭에 실어서 한 번에 옮기십시오.

□ 이사짐 옮기기 비유 (직관적 비유)

100만 개의 벽돌(데이터)을 옮겨야 합니다.

- **For-loop (Non-vectorized):** 인부가 벽돌을 손에 하나씩 들고 100만 번 왕복합니다. (파이썬이 데이터를 하나씩 꺼내서 처리함)
- **Vectorization:** 100만 개의 벽돌을 거대한 덤프트럭(행렬)에 싣고 단 한 번에 이동합니다. (NumPy가 데이터를 통째로 메모리에 올려 처리함)

기술적 정의: $z = w^T x + b$ 를 계산할 때, $w_1 x_1, w_2 x_2 \dots$ 를 순회하지 않고, w 와 x 전체 벡터를 한 번에 내적(Dot Product)하는 것입니다.

3.2 2. Under the Hood: SIMD (하드웨어의 마법)

왜 NumPy('np.dot')가 'for'문보다 빠를까요? 단순히 C언어로 짜여서가 아닙니다. 컴퓨터 구조적인 이유가 있습니다.

- **SISD (Single Instruction, Single Data):** 일반적인 'for'문입니다. CPU가 "가져와", "곱해", "저장해"를 데이터 하나마다 반복합니다.
- **SIMD (Single Instruction, Multiple Data):** 최신 CPU는 "이 8개의 데이터를 동시에 곱해!"라는 명령을 내릴 수 있습니다. 벡터화는 이 병렬 처리 기능을 활용합니다.

3.3 3. Broadcasting (브로드캐스팅)

한 줄 요약: 작은 행렬을 큰 행렬 크기에 맞게 자동으로 '늘려서(Stretch)' 연산합니다.

- **상황:** (4×1) 행렬에 숫자 100(스칼라)을 더하고 싶습니다.
- **원칙:** 수학적으로는 불가능하지만, Python은 100을 자동으로 (4×1) 크기로 복사하여 더해줍니다.

- 예시:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + 100 \rightarrow \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 100 \\ 100 \\ 100 \\ 100 \end{bmatrix} = \begin{bmatrix} 101 \\ 102 \\ 103 \\ 104 \end{bmatrix}$$

4 Implementation & Benchmark (성능 검증)

”백문이 불여일견”입니다. 100만 개의 데이터를 곱하는 시간을 직접 측정해 봅시다.

□ For-loop vs Vectorization 속도 대결 (실전 시나리오 & 벤치마크)

```

1 import numpy as np
2 import time
3
4 # 데이터 준비: 만100 개의 난수 생성
5 a = np.random.rand(1000000)
6 b = np.random.rand(1000000)
7
8 # --- 1. For-loop 느린( 방법) ---
9 c = 0
10 tic = time.time()
11 for i in range(1000000):
12     c += a[i] * b[i]
13 toc = time.time()
14
15 print(f"For-loop: {c:.4f}")
16 print(f"Time: {1000 * (toc - tic):.2f} ms") # 약 400~500ms 소요
17
18 # --- 2. Vectorization 빠른( 방법) ---
19 tic = time.time()
20 c_vec = np.dot(a, b) # SIMD 병렬 처리
21 toc = time.time()
22
23 print(f"Vectorized: {c_vec:.4f}")
24 print(f"Time: {1000 * (toc - tic):.2f} ms") # 약 1~2ms 소요

```

결과 분석: 벡터화 코드가 약 **300 500배** 더 빠릅니다. 딥러닝 모델 학습 시간이 1달 걸릴 것을 2시간으로 줄여주는 마법입니다.

5 Pitfalls & FAQ

Rank-1 Array의 함정 (오해 방지 가이드)

NumPy에서 ‘a = np.random.randn(5)’를 하면 모양(Shape)이 ‘(5,)’가 됩니다. 이것은 행 벡터도, 열 벡터도 아닌 애매한 상태라 전치(Transpose)가 안 됩니다.

- 나쁜 예: ‘a = np.random.randn(5)’ → 버그 발생 위험 높음.
- 좋은 예: ‘a = np.random.randn(5, 1)’ (열 벡터) 또는 ‘(1, 5)’ (행 벡터)로 명시하십시오.
- 습관: 코드 중간에 ‘assert(a.shape == (5, 1))’을 넣어 차원을 확인하십시오.

FAQ: 자주 묻는 질문

Q1. GPU는 언제 쓰나요?

A. NumPy는 기본적으로 CPU를 사용합니다. 나중에 배울 TensorFlow나 PyTorch는 이 벡터화 연산을 GPU(그래픽 카드)에서 수행하여, CPU보다 훨씬 더 많은 병렬 처리(수천 개의 코어)를 가능하게 합니다. 원리는 똑같습니다.

Q2. 모든 코드를 벡터화할 수 있나요?

A. 대부분의 수학 연산은 가능합니다. 하지만 복잡한 조건문('if-else')이 데이터마다 다르게 적용되어야 한다면 벡터화가 어려울 수 있습니다. 그럼에도 99%의 딥러닝 연산은 벡터화가 가능합니다.

다음 단계 (Next Step)

축하합니다! 여러분은 이제 '고속 연산 엔진(Vectorization)'을 장착했습니다.

지금까지는 뉴런이 딱 하나(로지스틱 회귀)뿐이었습니다. 다음 장 [Chapter 3. Shallow Neural Networks]에서는 이 강력한 엔진을 활용해 뉴런을 수백 개로 늘려보겠습니다. 이제 진짜 '신경망'다운 신경망을 만들 차례입니다.

□ 단원 요약 (Cheat Sheet)

1. **Vectorization:** 'for'문은 최악이다. 'np.dot' 등을 써서 행렬 단위로 계산하라.
2. **SIMD:** 벡터화는 CPU의 병렬 처리 명령어를 사용하여 속도를 수백 배 높인다.
3. **Broadcasting:** 차원이 달라도 NumPy가 알아서 맞춰주지만, 버그를 조심해야 한다.
4. **Shape Check:** '(n,)' 대신 '(n, 1)'을 사용하여 차원을 명시하는 습관을 들여라.